

F1R3COMB-MAT

The rho combinators as sparse linear algebra:
a Rust implementation of the matrix representation,
the bulk-synchronous machine, and the transfer operator
Specification v0.2 — companion to F1R3COMB v0.4

F1R3FLY.io

23 September 2026

Abstract

This is the second version of the specification for F1R3COMB-MAT, the arrow from a F1R3COMB term to the matrix representation of [1] and the bulk-synchronous machine that runs it: the route from the rho calculus, through the combinators, to a data-parallel device. Since v0.1, draft 3 has changed the translation it runs [3, §6]. Apparatus names are no longer positional, because positional names are unsound for code instantiated twice; a first phase compiles into the combinators with a binder, and a second eliminates the binder by allocating each run-time instance’s apparatus beneath an *address* it receives, and erecting its atoms there by a constructor followed by *e*. The four constructors have become a family, one per atom shape. F1R3COMB has followed, to v0.4 [2].

For a machine whose whole purpose is to run many things at once, the decisive part of this is phase two, and v0.2 is organised around it. Three findings, each backed by the reference implementation, shape the specification. First, concurrent instances erected by the literal construction [3, Def. 6.8] mix their apparatus in most runs, and maximal progress — the device’s native schedule — mixes more than sequential choice does ([1, Find. 14.1]: 58% of two-instance runs against 32%, every run from eight instances). Second, the discipline draft 3 proves sufficient for correctness [3, Def. 6.10, Lem. 6.11] cannot be met with fixed-arity constructors for any atom mentioning two allocated names — which includes the distributor and the gate — so the conforming construction F1R3COMB v0.4 mandates [2, Req. 5.16] does not exist [1, Prop. 14.3]. Third, a single-input, multi-hole constructor, which is draft 3’s own context-indexed alternative [3, Rem. 3.14] promoted to a primitive, meets the discipline by construction, erects an instance in two steps, keeps rows four wide, and runs the recursion combinator at a constant 7 parallel steps and 16 new names per unfolding however deep its address ([1, Find. 14.4]).

v0.2 therefore supports both erections: the literal one, because it is what F1R3COMB emits today and the machine must be able to show what it does, and single-input instantiation, behind a feature, as the construction the device path is built for. It generates its rule table from F1R3COMB’s shape table, makes the row width a per-artefact parameter, adopts F1R3COMB’s address allocator and freshness checks, adds an instance-provenance column for tracing and for a cross-instance resolver, and records that the operator layer’s closure applies to no compiled program with an input, since every one contains *e*. The three errata v0.1 raised against the notes are discharged by their revisions; the questions this version puts to the papers are about phase two. Key words MUST, MUST NOT, SHOULD, MAY are used in the sense of RFC 2119.

Contents

1	Purpose and scope	3
1.1	The route, and where this document sits on it	3
1.2	What exists today	4

1.3	Nomenclature	5
1.4	Goals	5
1.5	Non-goals	5
2	Position relative to F1R3COMB	6
2.1	Two machines, one term type	6
2.2	Decisions	6
3	Phase two, as the machine meets it	7
3.1	Concurrency is the normal case	7
3.2	The conforming construction does not exist	7
3.3	Single-input context instantiation	7
3.4	Provenance	8
4	The representation	8
4.1	Names, the intern table, and addresses	8
4.2	State	9
4.3	Import, export, and the canonical format	10
5	The rule table	10
5.1	Generation	10
5.2	The grading, generated	10
6	Sparse algebra and redex finding	11
7	Conflict, selection, and resolvers	11
7.1	Resolvers	11
8	Firing	12
9	Addresses and freshness	12
10	The operator layer, scoped	12
11	Instrumentation and the width gate	12
12	Architecture	13
12.1	Backends and the device seam	13
13	Correctness obligations and testing	14
14	Errors and diagnostics	14
15	Performance, portability, determinism	15
16	What the implementation asks of the papers	15
17	Open questions	16
18	Delivery	17
19	Changes from v0.1	17

1 Purpose and scope

1.1 The route, and where this document sits on it

The programme is a compiler from the rho calculus to a data-parallel device by way of the combinators and their matrix reading. It has four stages, and this document specifies the third.

Stage	What it does	Specified in
1	K0 text to a canonical normal form	[5]
2	Normal form to combinator term, in two phases	F1R3COMB v0.4 [2]
3	Combinator term to matrix representation; the bulk-synchronous machine; the transfer operator	this document
4	The machine's kernels on a device	Section 12.1, as a contract

Two properties are fixed from the start and govern everything below. The machine must compute exactly one step per state, seed and configuration, so that a host and a device can be compared trace for trace ([1, P4]); and it must run what stage two emits, including every instance of every unit, at the width the program has ([1, Princ. 10.4]).

1.2 What exists today

Artefact	What it gives this project
<code>rho-combinators-draft3</code>	The calculus and its two presentations (§3.2–3.3); the constructor family (Def. 3.8, Rem. 3.9); erection at a computed name (Rem. 3.11); the context-indexed constructor (Rem. 3.14); the positional-naming bug (§6.1); phase one (§6.2) and phase two (§6.3, Def. 6.6, Def. 6.8, Ex. 6.9); the discipline and the permutation lemma (Def. 6.10, Lem. 6.11, Rem. 6.12); freshness by size (Lem. 6.14); the weighted atom count (Lem. 9.2, Prop. 9.9)
<code>f1r3comb-spec-v04</code>	The term type and shape table; tags; the intermediate representation; phase two as a compiler; the address allocator; the golden terms W and D_x ; the diagnostics this document shares
<code>rhocomb-matrix v1.2</code>	The state space and incidence (§4, §7), the step (§8), the operator (§5–6), the plan (§11), and §14: phase two seen from the machine, the mixing measurements, the multiplicity proposition, context instantiation
<code>rhocombmat.py v1.2, experiments.py, phase2.py</code>	The reference machine with a generated rule table and the context constructor; E1–E4 (results.txt); the phase-two experiments (phase2-results.txt)
<code>rhocomb-logic rev. 2</code>	Components as a complete invariant (Def. 2.9); drop elimination (Lem. 2.8); injective quotation (Prop. 2.10); the weighted count as a formula (Def. 4.2, Prop. 6.17)
<code>campf1r3</code> (the repository)	The workspace; <code>Hash32</code> and <code>blake2b_256</code> ; no external dependencies

1.3 Nomenclature

Term	Meaning in this document
<i>term</i>	A <code>f1r3comb_term::Term</code> : a closed drop-free normal form under presentation A, a flat multiset of atoms [2, Req. 4.3]
<i>shape table</i>	F1R3COMB’s table of atom shapes with arities and argument sorts, from which constructors and rules are generated [2, Req. 4.5]
<i>family, $\mathcal{F}(P)$</i>	The constructors a program uses [2, Req. 4.7], recorded in the artefact header
<i>row width, w</i>	The largest arity in the program’s signature; the fixed width of every row in one run
<i>name id</i>	A <code>NameId(u32)</code> : the in-run index of a name modulo $\equiv_{\mathcal{N}}$
<i>row, row ref</i>	One atom occurrence; its position (<code>Shape, u32</code>) in its shape table
<i>address, leaf</i>	The name an instance receives and allocates beneath; a name $\sigma \cdot w$ so allocated [3, Def. 6.6]
<i>static channel</i>	A name in the phase-two image, shared by every instance [2, §1.3]
<i>unit, instance</i>	One quotation’s phase-two image; one run-time copy of it at one address
<i>provenance</i>	The instance a row belongs to, as far as the machine can tell; machine metadata, never semantics (Section 3.4)
<i>erection</i>	Building an atom at computed names: a constructor, then <code>e</code> [3, Rem. 3.11]; <i>literal</i> per [3, Def. 6.8], or by <i>instantiation</i> per Section 3.3
<i>redex, step, width</i>	As in v0.1: a rule with the row refs it consumes; one maximal independent set; its size

1.4 Goals

1. A total, lossless, round-trippable map from F1R3COMB terms to the matrix representation, and back, for every shape F1R3COMB’s shape table can generate.
2. Redex finding as joins derived from the generated rule table, with three independent finders that agree.
3. A bulk-synchronous step defined bit-exactly, so host and device traces are comparable.
4. Phase-two images run correctly: F1R3COMB’s address allocator and freshness checks, per-instance provenance, and both erections — literal, to test what F1R3COMB emits, and single-input instantiation, to build on.
5. The instruments that gate device work: width, with erection separated from computation; address growth; cross-instance mixing.
6. On the destructor-free sublattice, the transfer operator, its closure and the pre-image primitive, scoped honestly (Section 10).
7. F1R3COMB’s portability envelope: no external dependencies, `forbid(unsafe_code)`, WebAssembly-clean, bit-identical across hosts and core counts.

1.5 Non-goals

Device code (Section 12.1 is a contract, and gated); presentation B (Decision 2.1); weights; the logic; the history unfolding; intern-table collection; phase one and phase two themselves, which are F1R3COMB’s. F1R3COMB-MAT takes a `Term` and MUST NOT depend on `f1r3comb-lower` or `f1r3comb-ir`.

2 Position relative to F1R3COMB

2.1 Two machines, one term type

	F1R3COMB machine (<code>f1r3comb-run</code>)	F1R3COMB-MAT (<code>f1r3comb-par</code>)
Input	Term and artefact header	Term and artefact header
State	CAMPF1R3 store keyed by <code>Hash32</code>	shape tables of <code>NameId</code> rows, width w
Name identity	content hash of the encoding	structural intern in-run, content hash at the boundary
Event	one comm	one step, a maximal independent set
Resolver	store order key; an adversarial profile for testing [2, Obl. 11.2]	priority function; a cross-instance profile (Section 7.1)
Rules	generated from the shape table	generated from the same table
Addresses	allocator [2, Req. 9.5]	the same allocator
Output	a scene	a scene

2.2 Decisions

Decision 2.1 (Presentation A only). F1R3COMB-MAT implements presentation A, and under F1R3COMB’s `presentation-b` feature its crates compile to stubs returning `comb-mat-presentation`. F1R3COMB v0.4 withdrew one of its own reasons for A [2, Rem. 2.2]; this decision never rested on it. It rests on row width: under B a message payload is a process, rows are ragged, and (G1) of [1, §2.1] fails. The A/B comparison F1R3COMB requires [2, Obl. 11.8] is run on `f1r3comb-run`.

Decision 2.2 (A store row holds the quotation of its payload). Unchanged from v0.1, and now with a second dividend. The term type keeps $q(a, p)$ with a process payload [2, Dec. 2.1]; the row is $(q, id(a), id(@p))$, lossless because $@-$ is injective on normal forms [4, Prop. 2.10], and the shape tag carries the m/q distinction the logic needs. In the row representation the member $cons_q$ [3, Def. 3.8] is uniform with every other: its conclusion is the row (q, v_1, v_2) , and A’s coercion $@q(v_1, *v_2)$ [3, Rem. 3.9] never appears ([1, Rem. 3.1]). [2, Req. 4.8]’s requirement that the two readings agree holds by construction here and **MUST** be tested on `f1r3comb-run`’s side only.

Decision 2.3 (Decode by arena read). Unchanged from v0.1. [2, Req. 9.3] forbids the CAMPF1R3 machine to project a payload out of a name, because the short-cut is unsound under B. F1R3COMB-MAT is A-only and interns drop-free components, so its `eval` is an arena read, tested against the smart-constructed $*v$.

Decision 2.4 (The rule table is generated from F1R3COMB’s shape table). v0.1 asked for one rule table shared by both machines. F1R3COMB v0.4 goes further and generates it [2, Req. 4.5]. F1R3COMB-MAT **MUST** consume the same generator, living in `f1r3comb-term`, and **MUST NOT** list rules of its own. Join plans, rule classes, the grading and the row width are derived from the generated table (Section 5).

Decision 2.5 (Two erections, one of them behind a feature). F1R3COMB-MAT **MUST** run terms erected literally per [3, Def. 6.8], since that is what F1R3COMB emits, and **MUST** support the context constructor `inst` of Section 3.3 behind the feature `inst`, off by default until the paper settles phase two’s primitive (Obligation 13.8). The feature adds one shape and one rule; the rest of the machine is indifferent to it. The device path is specified for the `inst` construction (Section 12.1), because it is the one that is both correct under concurrency and fixed-width.

3 Phase two, as the machine meets it

This section fixes what F1R3COMB-MAT must do with a phase-two image, and why. The argument is in [1, §14]; what follows is its consequences as requirements.

3.1 Concurrency is the normal case

Two deploys of one program share every static channel, and so do two unfoldings of one recursive unit. Draft 3’s correctness rests on the permutation lemma [3, Lem. 6.11], which holds for images meeting the single-address-input discipline. The literal erection does not meet it [2, Req. 5.15], and [1, Find. 14.1] measures the consequence on the reference machine:

instances	2	4	8	16
maximal progress: runs with a mixed atom (of 50)	29	48	50	50
sequential uniform: runs with a mixed atom (of 50)	16	38	49	50
single-input instantiation, either scheduler	0	0	0	0

A bulk-synchronous machine fires every enabled constructor at once, so it exhibits the cross whenever two instances’ leaves are present together, which on a device is always. This is why F1R3COMB-MAT treats mixing as something to measure, not something the schedule might avoid.

3.2 The conforming construction does not exist

F1R3COMB v0.4 responds by mandating a two-stage erection: a single-input bootstrap that erects, at address-derived channels, erectors that may then join freely [2, Req. 5.16]. [1, Prop. 14.3] shows that under the discipline in the form the permutation lemma uses — no redex fired at a static channel draws occurrences of one instance’s address from two premises — no reachable atom contains the address twice. An erector that joins two leaves contains the address twice, and so does every atom mentioning two allocated names, which includes the distributor $\mathbf{d}(q_1, p_1, q_2)$ and the gate $\mathbf{s}(p_0, b, c)$. Stage (ii) of the two-stage scheme is therefore unreachable from stage (i), and draft 3’s [3, Rem. 6.12] describes no construction.

The obstruction is arity, not routing. A fixed-arity constructor places each input once, so building a name that mentions the address twice from one copy of it requires duplicating and rejoining, and the rejoin is the forbidden join. A constructor that places one input at many holes does not.

Hazard 3.1 (Correctness of compiled programs is gated). Until phase two has a primitive that meets the discipline, a compiled program that runs two instances of a unit concurrently has no correctness guarantee on any machine, and on this one fails in most runs. Every width measurement on compiled programs (Section 11) **MUST** be labelled with the erection used, and results obtained with literal erection **MUST** be treated as measurements of the schedule, not of the program.

3.3 Single-input context instantiation

Definition 3.2 (The context constructor). For a context C — a term with a hole in any number of name positions, including beneath quotation —

$$\mathbf{inst}(a, f, C) \mid \mathbf{m}(a, v) \longrightarrow \mathbf{m}(f, @C[v]).$$

A unit’s phase-two image under **inst** is $\mathbf{inst}(r, f, C_{\text{unit}}) \mid \mathbf{e}(f)$, where C_{unit} is the unit’s phase-one image with each bound name replaced by the leaf at a fixed-length path beneath the hole, and with the spare leaves posted for nested units and self-instantiation [2, Req. 5.11].

This is [3, Rem. 3.14]’s context-indexed constructor, with one premise. It meets the discipline by construction (one address input at the static r); permuting addresses among `inst` atoms is the harmless permutation of [3, Lem. 6.11]; and it removes the allocation tree, the per-node gadget channels [2, Req. 5.9], the fan-out count [2, Req. 5.12] and the constructors of constructors. Freshness is unchanged: leaves exceed the address in size, and the address and statics exceed every name of the image [3, Lem. 6.14], [2, Req. 5.22].

Requirement 3.3 (The context is a name with a reserved hole). C MUST be represented as an interned name in which a reserved `NameId::HOLE` stands for the hole. `HOLE` MUST have no counterpart in the term type other than inside the third argument of an `inst` atom, MUST be rejected by import anywhere else (`comb-mat-hole`), and MUST be unconstructible: no rule’s conclusion may contain it. In the term encoding it MUST take a tag in the reserved block, allocated by `F1R3COMB` (Section 16, item 4).

Requirement 3.4 (Instantiation is a memoised substitution). The encode of `inst` MUST be $C\{v/\text{HOLE}\}$ computed over the interned form, hereditarily beneath quotation, with a per-firing memo keyed by name id, so that its cost is the number of distinct names in C , not the size of C ’s tree. Its result names MUST be interned in canonical order (Requirement 8.1).

Requirement 3.5 (The measured cost is a regression). [1, Find. 14.4] runs D_x of [3, Ex. 6.9] under `inst`: exactly 7 parallel steps and 16 new names per unfolding for 200 unfoldings, and between 9 and 11 sequential steps (mean 10, the draft’s 2 erection steps plus 8). `f1r3comb-check` MUST reproduce these as laws over the number of unfoldings, to 10^4 unfoldings (Requirement 15.2).

3.4 Provenance

A trace of a phase-two run is unreadable without knowing which instance each atom belongs to [2, Req. 9.5], and a cross-instance resolver needs the same information. It is available to the machine without inspecting names.

Definition 3.6 (Provenance). Every row carries a provenance tag `Prov(u32)`: `STATIC` for rows imported from the artefact, and for the rest the tag of the root address beneath which the row’s allocated names lie, computed when the row is produced as follows. A row produced by a redex takes the tag of the redex’s premises if they agree on one non-`STATIC` tag, `STATIC` if all are `STATIC`, and `MIXED` otherwise; a row whose names include a leaf takes that leaf’s root’s tag, found in $O(1)$ from the intern table’s root column (Requirement 4.3).

Requirement 3.7 (Provenance is metadata). Provenance MUST NOT influence which redexes exist, which are selected under `MaximalProgress`, or what they produce. It MAY influence selection only under the resolver of Section 7.1 built for it. It MUST appear in traces, rendered as the root address’s content hash, and a row tagged `MIXED` MUST raise a trace event at level `traces` or above. Under `inst` no row SHOULD ever be tagged `MIXED`, and the test suite MUST assert it.

4 The representation

4.1 Names, the intern table, and addresses

```
#[derive(Copy, Clone, PartialEq, Eq, PartialOrd, Ord, Hash, Debug)]
pub struct NameId(pub u32);           // NameId::HOLE reserved (Requirement 3.3)

pub struct Row<const W: usize> { pub shape: ShapeId, pub args: [NameId; W], pub prov: Prov }
```

```

pub struct InternTable {
  arena: Vec<(ShapeId, SmallArgs)>, // components of every name, contiguous
  span: Vec<(u32, u32)>,           // NameId -> (offset, len)
  key:   HashMap<Box<[...]>, NameId, FixedHasher>,
  size:  Vec<u64>,                 // encoding size, for freshness by size
  root:  Vec<Option<NameId>>,       // the address a leaf lies beneath
  depth: Vec<u32>,                 // quotation depth, for canonical export
  chash: Vec<Option<Hash32>>,       // content hash, lazily
  budget: NameBudget,
}

```

Requirement 4.1 (Structural interning; the hot path is hash-free). As v0.1: a name’s key is its sorted component rows over child ids, **quote** and **drop** compute no BLAKE2b, and the content hash of [2, Req. 4.10] is computed only at the boundary. The arena stores rows of the name’s own width, so names built from shapes of different arity share one table.

Remark 4.2 (Structural interning is the prefix sharing F1R3COMB asks for). [2, Req. 9.7] asks that descending one address level cost $O(1)$ rather than $O(|\sigma|)$, and notes that without it a recursive program degrades quadratically [2, Req. 13.1]. Structural interning provides it by construction: $L\sigma$ is one new entry whose single component refers to σ ’s id, and [1, Find. 14.4] measures 16 new names per unfolding of D_x at every depth. The content hash, being of the full encoding, is $O(\text{depth})$ per new address; this is the reason, beyond v0.1’s, that F1R3COMB-MAT keeps it off the hot path, and the reason for item 7 of Section 16.

Requirement 4.3 (Size, root and depth columns). **size** MUST hold each name’s encoding length, computed from its children in $O(\text{components})$ at interning time, so that the freshness test by size [3, Lem. 6.14] is an integer comparison. **root** MUST hold, for a leaf, the root address beneath which it lies, computed at interning time from the child’s entry when the name has the L or R shape of [3, Def. 6.6]. Both columns are machine metadata subject to Requirement 4.4.

Requirement 4.4 (Ids never leave the process). As v0.1: no trace, report, diagnostic or export MAY contain a **NameId** or row position, except the canonically relabelled **.cmat**. Externally a name is its **Hash32**.

Requirement 4.5 (Budget). [2, Req. 9.7]’s budget: maximum distinct names, maximum components per name, maximum arena length, and maximum address depth. Exceeding any MUST halt the step before it commits, with **comb-name-budget** naming the rule and, for an address, the unit. Because every loop descends [2, Haz. 9.6], the budget is an operational parameter and MUST be reported with its high-water marks in every run report.

4.2 State

Requirement 4.6 (Structure of arrays, width w). A state is one table per shape in the program’s signature, each holding $\text{ar}(A)$ columns of **NameId** plus a provenance column. Column arrays are contiguous; the incidence M_j^A is the column array ([1, Def. 7.1]). The width w MUST be read from the artefact header, MUST be a const-generic parameter of the machine instantiated for $w \in \{4, 5, 6\}$, and an artefact whose signature needs more MUST be rejected with **comb-mat-width**. Under **inst** $w = 4$ for every program: the base shapes have arity at most three, **cons_d** and **cons_s** four, and **inst** three.

Remark 4.7 (Why the width is not a constant any more). Under literal erection phase two erects every phase-one atom that mentions a bound name, and the (o5) chains of phase one are constructor atoms on bound names, so a program with an (o5) occurrence needs **cons_{cons_d}**, of arity five with four premises ([1, §14.1]). F1R3COMB v0.4’s family computation includes these [2, Req. 4.7], but its tag table does not (Section 16, item 3). The two-stage erection, were it constructible, would add a third level.

4.3 Import, export, and the canonical format

Requirement 4.8 (Import and export). As v0.1, extended to the generated signature: import flattens components in encoding order and interns bottom-up with an explicit stack; export rebuilds through `f1r3comb-term`'s smart constructors; import-export is the identity on encodings and export-import on markings. Provenance is **STATIC** on import and is not exported.

Requirement 4.9 (`.cmat`). As v0.1, with the header extended by the fields `F1R3COMB`'s artefact header carries [2, §8.1]: presentation, feature set (including `inst`), erection construction, $\mathcal{F}(P)$, w , the root-address size bound, and the static channels. Canonical relabelling orders names by quotation depth then content hash; **HOLE** is relabelled to the reserved index 0. For terms $P \equiv Q$ the encodings **MUST** be byte-equal.

5 The rule table

5.1 Generation

```
pub enum Premise { Loud { slot: u8 }, Quiet { slot: u8 } }
pub enum Conclusion {
  Atoms(Vec<(ShapeId, Vec<Var>>>), // class (a)
  Encode { deliver: Var, node: Node }, // class (b): m(deliver, @node)
  Decode { payload: Var }, // class (c): *payload
}
pub enum Node { Union(Var, Var), Atom(ShapeId, Vec<Var>), Instantiate { ctx: Var, with: Var }
}
pub fn rules(shapes: &ShapeTable, features: Features) -> RuleTable; // in f1r3comb-term
```

Requirement 5.1 (What the generator emits). From the shape table the generator **MUST** emit: the seven routing and destructor rules and `quote` of [3, Def. 3.3]; `makei`; for each constructor `consA` in $\mathcal{F}(P)$, one rule with `ar(A)` loud premises at slots $0..ar(A)-1$ and conclusion `Encode{deliver: Slot(ar(A)), node: Atom(A, payloads)}`; and under `inst` the rule of Definition 3.2, one loud premise at slot 0 and conclusion `Encode{deliver: Slot(1), node: Instantiate{ctx: Slot(2), with: Payload(0)}`. Rule classes, join plans, the grading and w **MUST** be derived from the emitted table.

Requirement 5.2 (Only a forwarder reads a store). As v0.1, and now a property of the generator: no generated rule other than `quote` **MAY** have a quiet premise. `consq` *builds* a store name and reads only messages. A test **MUST** assert, for every generated consumer shape other than `fw`, that it has no redex against a `q` row.

Requirement 5.3 (Classes and plans are checked). A test **MUST** check, for the generated table of every corpus artefact: exactly one class-(c) rule; one class-(b) rule per member of $\mathcal{F}(P)$, plus `makei` and, under `inst`, `inst`; seven class-(a) rules; and a join plan per rule whose arity equals its premise count. This replaces v0.1's hand-stated count, which the family made obsolete.

5.2 The grading, generated

Requirement 5.4 (Grading). The weight W of [3, Lem. 9.2] — `bl`, `br`, `s` at two, `m` at zero, every other non-message shape at one — **MUST** be derived from the table as in v0.1: one plus the heaviest non-message atom any of a shape's rules produces from it. For the family and for `inst` this gives one. The debug build **MUST** assert that every non-`eval` firing decreases W , and that without `d` or `e` firings a step does not increase messages plus stores ([1, Thm. 6.1(iii)]).

6 Sparse algebra and redex finding

The sparse layer (**Semiring**, **Csr**, Gustavson **spgemm**, counting-sort **bucket**) is unchanged from v0.1, as are the three finders (**Naive**, **Product**, **Bucketed**), the canonical key, the distinctness and symmetry masks, and the requirement that all three return identical sorted redex sets. Two things change.

Requirement 6.1 (Joins are n -way). A join plan has up to $w - 1$ premises. The **Product** finder MUST compute one product per premise slot and an n -fold row-wise expansion with the masks; the **Bucketed** finder MUST look up one bucket per slot. Neither MAY special-case arity. The redex type is (rule: u16, consumer: u32, premises: [u32; W-1]).

Requirement 6.2 (Validation covers the generated signature). Finder agreement (Obligation 13.2) MUST be run over every shape the generator can emit for the corpus, two levels of the family, and **inst** — the reference’s version 1.2 validation does this over twenty-nine shapes ([1, Find. 10.1]).

Hazard 6.3 (Contention at static channels). v0.1 flagged the quadratic join on a contended channel ([1, Rem. 7.2]). Phase two makes static channels contended by design: n instances put n messages at each allocation channel under literal erection, and at r under **inst**. Under **inst** the contended rule has one premise and the redex count at r is n^2 , whose maximal independent set is a perfect matching, as in E2; under literal erection the multi-premise constructors at static channels enumerate n^m candidates for an m -input member. The enumerated-to-fired ratio MUST be reported per static channel.

7 Conflict, selection, and resolvers

The conflict hypergraph, the priority function, the greedy maximal independent set and the proposition that greedy selection equals parallel rounds under shared priorities are unchanged from v0.1 (**mix64** of **SplitMix64** over seed, step and canonical key; [6]). So is the requirement that every backend compute exactly the greedy set.

7.1 Resolvers

```
pub enum Resolver {
    MaximalProgress,    // the greedy maximal step; the default
    SingleByPriority,    // the first redex in priority order; linearises a step
    SingleUniform,      // one redex, uniformly over the (symmetry-masked) redex set
    CrossInstance,      // maximal progress, preferring redexes that join provenances
}
```

Requirement 7.1 (The cross-instance resolver). **CrossInstance** MUST compute the greedy maximal independent set under the priority $(\neg x(r), h(r), \text{key}(r))$, where $x(r)$ holds iff the premises of r carry two distinct non-**STATIC** provenance tags. It is the matrix machine’s counterpart of the adversarial profile [2, Obl. 11.2] requires of **CAMPF1R3**, and is stronger: it prefers every cross at every static channel in the same step. It MUST be used in the phase-two tests of Section 13 alongside **MaximalProgress**, and a term that passes **F1R3COMB**’s conformance check [2, Req. 5.14] yet yields a **MIXED** row under it MUST be reported as a counterexample to the conformance check, not patched.

Requirement 7.2 (Races are visible). As v0.1, with [2, Req. 9.8]’s third race added: every selected redex at a static channel whose excluded rivals carry a different provenance MUST be a trace event at level **races**, naming both addresses by content hash.

8 Firing

Requirement 8.1 (Order of effects). As v0.1: sort the step by canonical key; compute conclusions in that order, class (b) interning new names in that order — including the names an **inst** substitution creates, in the order of a fixed traversal of C — and class (c) reading the arena; remove consumed rows by stable compaction; append produced rows in the order computed. Parallel backends MAY compute concurrently but MUST assign ids as if sequentially. Budget checks precede any mutation.

Requirement 8.2 (Soundness is checked, not assumed). As v0.1: the debug build re-finds redexes after each firing of a step in canonical order and checks the rest remain enabled.

9 Addresses and freshness

Requirement 9.1 (One allocator for both machines). The address allocator of [2, Req. 9.5, Req. 5.23] MUST be factored out of **f1r3comb-run** into **f1r3comb-term** (module **addr**) and used unchanged by F1R3COMB-MAT, so that a deploy on either machine receives the same root address for the same program and allocator state. Two machines that disagree on addresses cannot be compared trace for trace.

Requirement 9.2 (Deploy checks). Deploy MUST perform F1R3COMB’s three freshness checks, with its diagnostics: the root address exceeds, by encoding size, every name of the image (**comb-address-too-small**, [2, Req. 5.19]); so does every static channel (**comb-static-not-fresh**, [2, Req. 5.22]); and the root address is prefix-incomparable with every address in use (**comb-address-overlap**, [2, Req. 5.23]). Size comparison uses the **size** column; prefix comparison walks the **root** and L/R spine and is linear in depth.

10 The operator layer, scoped

The operator layer of v0.1 — species, markings, successors with mass-action coefficients, exploration, the materialised T , triangularity in the marking order graded by W , closure by repeated squaring, and $T^\top \chi$ — is unchanged in content. Its scope is restated.

Requirement 10.1 (Scope, stated in the tool). Every phase-two image contains **e** (each gate stores its body behind one [3, Lem. 6.2], and each erection ends in one), so **explore** MUST refuse every compiled program with an input with **comb-op-destructor** unless given a depth bound, and the documentation MUST say that triangularity and closure ([1, Thm. 6.1(ii), Cor. 6.2]) apply to hand-written terms and to phase-one images of input-free programs only ([1, §14.2]). The **successors** primitive is unrestricted and is what the logic’s modality needs at a frontier.

Remark 10.2 (F1R3COMB’s open question on the matrix encoding). [2, §15, item 5] observes that an upper-triangular encoding wants a bounded name universe and that phase two grows it with every unfolding. The answer, from this side: the machine never wanted a bounded universe — it multiplies the term-sized incidence, not the state-indexed operator ([1, §7]), and its per-unfolding name cost is constant — and the closure, which does want one, is unavailable to compiled programs with inputs for the independent reason that they contain **e**.

11 Instrumentation and the width gate

Requirement 11.1 (Reports). As v0.1 (steps, rewrites, width and enumeration histograms, names, arena, inert rows, per-rule counts), plus: steps and rewrites attributed to *erection* — **inst** and constructor firings whose conclusion is consumed by **e** in the next step, and those **e** firings

— separately from the rest; instances started; address depth high-water mark; new names per instance; and the count of MIXED rows.

Requirement 11.2 (The width measurement). `f1r3x comb-width` MUST report, per program and per erection, the width distribution with erection separated from computation, as [1, Obl. 14.6] asks. Under `inst`, erection is two steps per instance and parallel across instances, so it should add width and not depth; the report is where that is confirmed or refuted. This is the instrument for [1, Obl. 13.7], which [1, Obl. 14.5] now gates.

Requirement 11.3 (Subcommands). F1R3X gains `comb-mat`, `comb-run -machine=matrix` (with `--resolver`, `--erection`, `--seed`, `--cores`, `--trace`), `comb-width` and `comb-op`, alongside F1R3COMB’s `comb`, `comb-ir`, `comb-hash`, `comb-run` and `comb-lattice`.

12 Architecture

Crate	What it is	Depends on
<code>f1r3comb-term</code>	(F1R3COMB’s) plus the rule generator (Decision 2.4) and the address allocator (Requirement 9.1)	<code>k1nd11ng-norm</code>
<code>f1r3comb-mat</code>	Intern table with size, root and depth columns; rows of width w ; shape tables with provenance; import, export, <code>.cmat</code> ; sparse layer	<code>f1r3comb-term</code>
<code>f1r3comb-par</code>	Join plans, finders, conflict, priority, resolvers, firing (including <code>inst</code>), deploy checks, backends, reports	<code>f1r3comb-mat</code>
<code>f1r3comb-op</code>	Species, markings, successors, exploration, T , closure, <code>pre</code>	<code>f1r3comb-par</code>
<code>f1r3comb-check</code> (shared)	Python parity, trace certification, phase-two tests, differential tests against <code>f1r3comb-run</code> and B3LL0WS	all

Requirement 12.1 (Independence). As v0.1, and in particular: none of the three crates MAY depend on `f1r3comb-ir`, `f1r3comb-lower` or `f1r3comb-run`.

12.1 Backends and the device seam

The `Backend` trait, the sequential and threaded host backends, and the rule that device backends live outside the workspace are unchanged from v0.1. The kernel contract is restated for the generated table.

Requirement 12.2 (Device kernels). A device backend MUST implement, against the `.cmat` layout: bucketing as a sort by key; one segmented n -way expansion per join plan with the masks; `mix64` priorities; the rounds of the greedy-equals-rounds proposition; stable compaction and a prefix-sum append; class (b) as a batched insert-or-get; class (c) as a gather from the arena; and, under `inst`, substitution as a gather over the context’s interned form with a batched insert-or-get of the results. It MUST target $w = 4$ and the `inst` erection (Decision 2.5); a device backend for literal erection MUST NOT be built, because under concurrency it computes wrong answers (Hazard 3.1). It is accepted only on identical step traces to `HostSequential` on the conformance corpus, and no work on it MAY begin before Obligations 13.8 and 13.7 are discharged.

13 Correctness obligations and testing

Obligation 13.1 (Carried from v0.1). Round trip; finder agreement over the generated signature (Requirement 6.2); Python parity on E1–E4, exact against `results.txt`; trace certification by an independent single-redex rewriter written from [2, Def. 4.4]; operator correctness on destructor-free terms; determinism across backends, core counts and WebAssembly; the invariants of Requirement 5.4; device conformance corpus.

Obligation 13.2 (Finder agreement). Requirement 6.2: on at least 10^4 random states over every generated shape, the three finders agree exactly.

Obligation 13.3 (The W regression, on this machine). [2, Obl. 11.1]’s term W and its three-instance variant, compiled by F1R3COMB, run under F1R3COMB-MAT with `MaximalProgress` and `CrossInstance`, for 100 seeds each, under each erection. Under `inst` the observation at $\llbracket @X \rrbracket$ MUST match B3LL0WS’s on every seed and no row MUST be `MIXED`. Under literal erection the test MUST record the fraction of seeds producing the crossed residue; a fraction of zero under `CrossInstance` would contradict [1, Find. 14.1] and MUST be investigated.

Obligation 13.4 (The phase-two experiments, reproduced). `f1r3comb-check` MUST reproduce `phase2-results.txt` in kind: the mixing counts of [1, Find. 14.1] are seed-dependent and are compared as rates within a tolerance; the D_x laws of Requirement 3.5 are exact.

Obligation 13.5 (Addresses). [2, Obl. 11.5, Obl. 11.10] on this machine: the three deploy checks, including the adversarial source term $@(x!(0))$ whose encoding is a leaf of $\llbracket @x \rrbracket$; and address independence, by running the corpus at two unrelated root addresses and comparing observations.

Obligation 13.6 (Differential testing). For programs whose observations are resolver-independent, F1R3COMB-MAT, `f1r3comb-run` and B3LL0WS MUST agree on the observation of [2, Obl. 11.7]. Under `inst` this requires F1R3COMB to emit the `inst` erection (Section 16, item 2); until it does, F1R3COMB-MAT compiles `inst` images from F1R3COMB’s phase-one output with its own test-only emitter in `f1r3comb-check`, which MUST NOT be linked into any product crate.

Obligation 13.7 (The width of real programs). [1, Obl. 13.7], under `inst`, with erection separated. Gates the device seam.

Obligation 13.8 (Phase two’s primitive). [1, Obl. 14.5]: the paper decides phase two’s primitive. Gates correctness of compiled programs under concurrency, hence the device seam.

14 Errors and diagnostics

Shared with F1R3COMB [2, §12]: `comb-feature-mismatch`, `comb-presentation-mismatch`, `comb-name-budget`, `comb-address-too-small`, `comb-static-not-fresh`, `comb-address-overlap`, `comb-decode`. Specific to F1R3COMB-MAT:

Code	Where	Meaning
<code>comb-mat-presentation</code>	any	Built or invoked under <code>presentation-b</code>
<code>comb-mat-width</code>	import	The signature needs rows wider than the instantiated w
<code>comb-mat-hole</code>	import, step	<code>HOLE</code> outside the context argument of an <code>inst</code> atom, or in a conclusion
<code>comb-mat-hash-collision</code>	boundary	Structural key and content hash disagree
<code>comb-mat-overflow</code>	sparse, op	Checked <code>Nat</code> arithmetic
<code>comb-mat-decode</code>	import	Malformed <code>.cmat</code>
<code>comb-op-destructor</code>	op	Exploration of a term that can reach <code>e</code> , without a depth bound; every compiled program with an input
<code>comb-op-budget</code>	op	Exploration budget exceeded
<code>comb-mat-internal</code>	any	An invariant assertion failed; names the requirement

15 Performance, portability, determinism

Requirement 15.1 (Complexity). As v0.1; in addition a step MUST NOT depend on address depth, which structural interning secures, and an `inst` firing MUST cost $O(|C|_{\#})$ where $|C|_{\#}$ is the number of distinct names in the context.

Requirement 15.2 (Targets). v0.1’s budget figures, plus: 10^4 unfoldings of D_x under `inst` with no rise in per-unfolding step time, which is [2, Req. 13.2]’s test of prefix sharing met by construction; and 64 concurrent instances of a 10^3 -atom unit erected in two steps.

Requirement 15.3 (Portability and determinism). As v0.1 and [2, Req. 13.3]: `wasm32-unknown-unknown`, `forbid(unsafe_code)`, no external dependency, no `RandomState`; bit-identical traces, reports and exports across runs, hosts, core counts and WebAssembly, given the same term, root address, seed, resolver, erection and configuration.

16 What the implementation asks of the papers

Discharged since v0.1. The atom-count lemma is weighted and the length bound restated [3, Lem. 9.2, Prop. 9.9]; the matrix note’s nilpotency index, closure and message-plus-store invariant are corrected ([1, Thm. 6.1, Cor. 6.2]); the reference counts a symmetric match once; the rules of the shortcut constructors are now members of the family [3, Def. 3.8]; the rule table is generated [2, Req. 4.5]; the `consq` reconciliation v0.1 asked for is [3, Rem. 3.9], and is trivial in the row representation.

Open.

1. **Draft 3, Definition 6.10 and Remark 6.12.** The discipline is stated for constructors, but a router holding a leaf at a static subject joins instances just as well ($\text{fw}(k, \text{L}\sigma) \mid \text{m}(k, \text{L}\tau)$), so the proof of Lemma 6.11 needs the discipline for every redex ([1, Def. 14.2]). In that form it cannot be met by fixed-arity constructors for any atom mentioning two allocated names ([1, Prop. 14.3]), so Remark 6.12 describes no construction, and Remark 3.14’s claim that the context-indexed constructor has “the same reach” as the family is true of reachability but false under the discipline. Proposed amendment: promote the context-indexed constructor, in its single-input multi-hole form, to a primitive, and state phase two with it. **GATE**

2. **F1R3COMB v0.4, Requirement 5.16.** The two-stage erection’s stage (ii) is unreachable from stage (i) by the same proposition, so the requirement mandates a construction that does not exist. Proposed amendment: replace it with the **inst** erection of Definition 3.2, which removes Requirements 5.9, 5.12 and 5.16 and the constructors of constructors, and keeps 5.11, 5.19–5.23 unchanged.
3. **F1R3COMB v0.4, the tag table of Requirement 4.11.** The constructor flag covers constructors of base shapes, but under literal erection $\mathcal{F}(P)$ contains $\mathbf{cons}_{\mathbf{cons}_A}$ whenever the program has an (o5) occurrence, and those have no tag. Moot under item 2; otherwise the layout needs a level field.
4. **F1R3COMB v0.4, tags for the context constructor.** Under item 2, **inst** needs a shape tag and the hole needs a reserved name tag outside the source block (Requirement 3.3).
5. **F1R3COMB v0.4, Requirement 9.5.** It asks for “a root address of a shape outside that program’s family”, which is v0.3’s shape-based freshness; Requirement 5.19 and draft 3’s Lemma 6.14 are by size. Proposed amendment: restate 9.5 by size.
6. **F1R3COMB v0.4, citations to the logic note.** The weighted count is cited as [4]’s “Def. 6.6, Prop. 9.2”; it is Definition 4.2 and Proposition 6.17 there, and [3, Lem. 9.2] in the companion.
7. **F1R3COMB v0.4, Requirements 9.7 and 13.1.** Prefix sharing makes *interning* $O(1)$ per level, but the content hash of the full encoding remains $O(\text{depth})$, and CAMPF1R3 keys channels by it. Proposed: a Merkle form of the canonical hash (tag, then child hashes) for names, which makes a new address’s hash $O(1)$ and keeps equality of hashes equivalent to $\equiv_{\mathcal{N}}$; it is a change to [5]’s hashing and wants deciding there.
8. **F1R3COMB v0.4, Requirement 9.3** (decode by projection), unchanged from v0.1: scope it to builds where **presentation-b** is reachable (Decision 2.3).
9. **F1R3COMB v0.4, the address allocator.** Move it to **f1r3comb-term** so both machines share it (Requirement 9.1).

17 Open questions

1. **The primitive** (Obligation 13.8): the single-input context constructor, or another construction meeting the discipline.
2. **Width under instantiation.** Whether erection, at two steps per instance, adds width without depth on real programs.
3. **Resolver fidelity and fairness** ([1, Obl. 13.1, Obl. 13.6]); now also: which resolver a correct phase-two image is insensitive to, which the permutation lemma says is all of them.
4. **Contention at statics** (Hazard 6.3): under **inst** the quadratic join sits at r , one per unit; whether bucketed matching at a one-premise channel should bypass enumeration.
5. **Address compaction** [2, §15, item 1]: whether a retired subtree may be reused, which the intern table would make cheap and the permutation lemma would have to license.
6. **Garbage, the history unfolding, weights:** as v0.1.

18 Delivery

M	Deliverable	Done when
X0	<code>f1r3comb-mat</code>	Intern table with size, root and depth columns; rows of width w ; provenance; import, export, <code>.cmat</code> ; round trip. Needs F1R3COMB M0 and the rule generator in <code>f1r3comb-term</code>
X1	Finders	n -way plans from the generated table; three finders; masks; agreement over the generated signature
X2	<code>f1r3comb-par</code> , sequential	Resolvers including CrossInstance ; firing including inst ; deploy checks with the shared allocator; Python parity; certification; invariants; the D_x laws
X3	Phase-two evidence	The W regression under both erections and both resolvers; the mixing rates of [1, Find. 14.1]; reported to the paper as evidence for Obligation 13.8
X4	The width gate	Under inst , with erection separated; needs F1R3COMB M2 or the test-only emitter. Gate on X7
X5	Threaded host	Determinism across cores and wasm32 ; differential tests
X6	<code>f1r3comb-op</code>	As v0.1, scoped per Requirement 10.1
X7	Device conformance	Corpus under inst ; external backend only after Obligations 13.8 and 13.7

X3 is new and comes early on purpose. The machine is the cheapest instrument there is for the question phase two now hinges on, because maximal progress and **CrossInstance** produce, as a matter of course, the interleavings a store-based machine must be coaxed into.

19 Changes from v0.1

v0.1	v0.2
Thirteen shapes, fifteen with shortcuts; fixed width four	Generated signature; width w per artefact, four under inst
One rule table, shared	Generated from F1R3COMB's shape table
Shortcut constructor rules fixed here	Members of the family
No addresses	F1R3COMB's allocator, shared; size, root and depth columns; deploy checks; provenance
Three resolvers	Four; CrossInstance added
—	Section 3: phase two as the machine meets it; the inst erection behind a feature; Hazard 3.1
Operator layer for the e -free sublattice	The same, with its scope stated: no compiled program with an input
Three errata against the notes	Discharged by their revisions
Eleven asks of the papers	Nine, all but one new, the first a gate
Delivery X0–X6	X0–X7, with the phase-two evidence at X3

References

- [1] L. G. Meredith. *Rho combinators as sparse linear algebra*. Version 1.2, `publications/rho-combinators/rhocomb-matrix`, with `rhocombmat.py`, `experiments.py` and `phase2.py`, September 2026.
- [2] F1R3FLY.io. *F1R3Comb: a compiler from kernel rho to the name-free rho combinators, and a machine that runs the result*. Specification v0.4, `publications/rho-combinators/f1r3comb-spec-v04`, 23 September 2026.
- [3] L. G. Meredith and M. Stay. *Name-free combinators for the rho calculus: a repair, a linear encoding, and the one operation that was missing*. Draft 3, `publications/rho-combinators`, September 2026.
- [4] F1R3FLY.io. *A spatial-behavioural logic for the rho combinators*. Draft 2, revision 2, `publications/rho-combinators/rhocomb-logic`, 23 September 2026.
- [5] F1R3FLY.io. *K1ndl1ng: a parser and normaliser for kernel rholang*. `publications/campf1r3/k1ndl1ng-spec`, 2026.
- [6] G. E. Blelloch, J. T. Fineman and J. Shun. Greedy sequential maximal independent set and matching are parallel on average. *SPAA*, 2012.
- [7] F. G. Gustavson. Two fast algorithms for sparse matrices: multiplication and permuted transposition. *ACM Transactions on Mathematical Software*, 1978.
- [8] G. L. Steele, D. Lea and C. H. Flood. Fast splittable pseudorandom number generators. *OOPSLA*, 2014.